FoundIt — Lost and Found System

Week 4: Queries and Full System Integration

Class Code **4-311** · Group **2** · 7 November 2026

1. Query Summary

Eight information-retrieval queries are exposed from the Java console application under **Main Menu → 5. Reports / queries**. Each one is implemented as a `PreparedStatement` in `QueryMenu.java`.

#	Business Question	SQL Feature
1	Which items belong to a given category?	`WHERE`
2	Where was each item found or lost, and who reported it?	`JOIN`
3	How many items has each reporter submitted?	`JOIN + COUNT + GROUP BY`
4	Which found items have never been claimed?	`Subquery`
5	How many claims has each validator verified versus rejected?	`JOIN + SUM + GROUP BY`
6	Which items have more than one claim request filed against them?	`GROUP BY + HAVING`
7	What are the five most recently found items?	`ORDER BY`
8	How long does it take, on average, from found to claim, per category?	`AVG + GROUP BY`

Table 1. Query list, business question, and the SQL feature demonstrated.

2. SQL and Query Results

Every query below was executed through the Java menu against the `foundit_db` database. The monospace blocks are the console output produced by the application.

Query 1 — Items in a category (WHERE)

```
SELECT item_id, item_name, item_category, item_type FROM Item WHERE item_category = ?
```

Result **rows returned**

```
item_id                  item_name                item_category            item_type
-------------------------------------------------------------------------------------------------------
3                        Scientific Calculator    Electronics              FoundItem
5                        Wireless Earbuds Case    Electronics              FoundItem
10                       Laptop Charger           Electronics              LostItem
```

Query 2 — Items with location and reporter (JOIN)

```
SELECT i.item_id, i.item_name, i.item_type, l.location_name, CONCAT(u.first_name,' ',u.last_name) AS reporter FROM Item i JOIN Location l ON l.location_id =
i.location_id JOIN User u ON u.user_id = i.reporter_id ORDER BY i.item_id
```

Result **rows returned**

```
item_id                  item_name                item_type                location_name            reporter
-------------------------------------------------------------------------------------------------------------------------------
1                        Black Umbrella           FoundItem                University Canteen       AJ Castro
2                        Blue Tumbler             FoundItem                Gisbert Library          Jose Ramirez
3                        Scientific Calculator    FoundItem                Finster Hall Lobby       Angela Reyes
4                        Student ID with Lanyard  FoundItem                Main Gate Guardhouse     AJ Castro
5                        Wireless Earbuds Case    FoundItem                Computer Laboratory 2    Paolo Contis
6                        Gray Hoodie              FoundItem                Covered Court            Jose Ramirez
7                        Brown Leather Wallet     FoundItem                University Canteen       Angela Reyes
8                        House Keys               FoundItem                Finster Hall Lobby       Paolo Contis
9                        Black Backpack           LostItem                 Gisbert Library          AJ Castro
10                       Laptop Charger           LostItem                 Computer Laboratory 2    Jose Ramirez
11                       Eyeglasses               LostItem                 Covered Court            Angela Reyes
12                       Blue Spiral Notebook     LostItem                 Finster Hall Lobby       Paolo Contis
```

Query 3 — Items reported per reporter (JOIN + COUNT + GROUP BY)

```
SELECT u.user_id, CONCAT(u.first_name,' ',u.last_name) AS reporter, COUNT(i.item_id) AS items_reported FROM User u JOIN Item i ON i.reporter_id = u.user_id GROUP BY
u.user_id, u.first_name, u.last_name ORDER BY items_reported DESC
```

Result **rows returned**

```
user_id                     reporter                    items_reported
-------------------------------------------------------------------------------
1                           AJ Castro                   3
2                           Jose Ramirez                3
3                           Angela Reyes                3
4                           Paolo Contis                3
```

Query 4 — Found items never claimed (Subquery)

```
SELECT i.item_id, i.item_name, f.found_at FROM Item i JOIN FoundItem f ON f.item_id = i.item_id WHERE i.item_id NOT IN (SELECT item_id FROM ClaimRequest)
```

Result **0 rows**

```
item_id                     item_name                   found_at
-------------------------------------------------------------------------------
(no results)
```

Query 5 — Verified/unverified claims per validator (JOIN + SUM + GROUP BY)

```
SELECT u.user_id, CONCAT(u.first_name,' ',u.last_name) AS validator, SUM(c.request_type = 'VerifiedRequest') AS verified, SUM(c.request_type = 'UnverifiedRequest')
AS unverified FROM ClaimRequest c JOIN User u ON u.user_id = c.validator_id GROUP BY u.user_id, u.first_name, u.last_name
```

Result **rows returned**

```
user_id                     validator                   verified                    unverified
-------------------------------------------------------------------------------------------------------------
10                          Ramon Aquino                2                           1
11                          Stephanie Domingo           2                           2
12                          Anika Fernandez             1                           2
```

Query 6 — Items with more than one claim (GROUP BY + HAVING)

```
SELECT i.item_id, i.item_name, COUNT(*) AS claim_count FROM ClaimRequest c JOIN Item i ON i.item_id = c.item_id GROUP BY i.item_id, i.item_name HAVING COUNT(*) > 1
```

Result **rows returned**

```
item_id                     item_name                   claim_count
--------------------------------------------------------------------------------
2                           Blue Tumbler                2
6                           Gray Hoodie                 2
```

Query 7 — Five most recently found items (ORDER BY)

```
SELECT i.item_id, i.item_name, f.found_at FROM FoundItem f JOIN Item i ON i.item_id = f.item_id ORDER BY f.found_at DESC LIMIT 5
```

Result **rows returned**

```
item_id                     item_name                   found_at
--------------------------------------------------------------------------------
8                           House Keys                  2026-10-05 09:30:00
7                           Brown Leather Wallet        2026-10-01 13:15:00
6                           Gray Hoodie                 2026-09-23 18:00:00
5                           Wireless Earbuds Case       2026-09-21 16:20:00
4                           Student ID with Lanyard     2026-09-18 07:45:00
```

Query 8 — Average hours from found to claim, per category (AVG + GROUP BY)

```
SELECT i.item_category, ROUND(AVG(TIMESTAMPDIFF(HOUR, f.found_at, c.request_date)),1) AS avg_hours_to_claim FROM ClaimRequest c JOIN FoundItem f ON f.item_id =
c.item_id JOIN Item i ON i.item_id = c.item_id GROUP BY i.item_category ORDER BY avg_hours_to_claim
```

Result **rows returned**

```
item_category               avg_hours_to_claim
----------------------------------------------------------
Drinkware                   11.0
Umbrella                    20.0
Wallet/Bags                 20.0
Electronics                 22.0
ID/Documents                24.0
Keys                        24.0
Clothing                    175.0
```

3. Java Implementation

3.1 Application structure

Class	Responsibility
`Foundit`	Entry point and main menu; dispatches to each menu class
`Database`	Prompts for the password, opens the connection, rolls back failed transactions
`Console`	Shared prompt/parse helpers (`readLine` , `readInt` , `readChoice`)
`ViewMenu`	Lists rows of User, Location, Item, ClaimRequest, ItemPhoto
`InsertMenu`	Inserts, writing parent and child tables in one transaction
`UpdateMenu`	Updates location, item, user contact, rejection reason
`DeleteMenu`	Deletes child rows first, then the parent row
`QueryMenu`	The eight report queries in Table 1

3.2 Main menu

```
private static void runMainMenu(Connection conn, Console console) {
        boolean running = true;

        while (running) {
            System.out.println("\n===== FoundIt Lost and Found System =====");
            System.out.println("1. View records");
            System.out.println("2. Insert record");
            System.out.println("3. Update record");
            System.out.println("4. Delete record");
            System.out.println("5. Reports / queries");
            System.out.println("6. Exit");

            try {
                conn.setAutoCommit(true);

                switch (console.readChoice("Choose: ")) {
                    case 1 -> new ViewMenu(conn, console).run();
                    case 2 -> new InsertMenu(conn, console).run();
                    case 3 -> new UpdateMenu(conn, console).run();
                    case 4 -> new DeleteMenu(conn, console).run();
                    case 5 -> new QueryMenu(conn, console).run();
                    case 6 -> {
                        System.out.println("Goodbye!");
                        running = false;
                    }
                    default -> System.out.println("Invalid choice. Please enter 1-6.");
                }
            } catch (SQLException e) {
                System.out.println("Error: " + e.getMessage());
                Database.rollbackIfNeeded(conn);
            } catch (NumberFormatException e) {
                System.out.println("Invalid number. Please enter digits only.");
            } catch (IllegalArgumentException e) {
                System.out.println("Invalid date. Use the format yyyy-MM-dd HH:mm:ss");
            }
```

```
        }
    }
```

Foundit.java — reports are reached through menu option 5.

3.3 Query implementation

```
public void run() {
        while (true) {
            System.out.println("\n===== REPORTS / QUERIES =====");
            for (int i = 0; i < TITLES.length; i++) {
                System.out.printf("%d. %s%n", i + 1, TITLES[i]);
            }
            System.out.println("0. Back to main menu");

            int choice;
            try {
                choice = Integer.parseInt(console.readLine("Choose: "));
            } catch (NumberFormatException e) {
                System.out.println("Invalid input. Enter a number from 0 to " + TITLES.length + ".");
                continue;
            }

            if (choice == 0) {
                return;
            }
            if (choice < 1 || choice > TITLES.length) {
                System.out.println("Invalid choice. Enter a number from 0 to " + TITLES.length + ".");
                continue;
            }
            runQuery(choice - 1);
        }
    }

    private void runQuery(int idx) {
        try (PreparedStatement ps = conn.prepareStatement(SQL[idx])) {
            if (idx == 0) {
                String category = console.readLine("Category (e.g. Electronics, Clothing, Keys): ");
                if (category.isEmpty()) {
                    System.out.println("Category cannot be empty.");
                    return;
                }
                ps.setString(1, category);
            }

            try (ResultSet rs = ps.executeQuery()) {
                System.out.println("\n-- " + TITLES[idx] + " --");
                print(rs);
            }
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }

    private void print(ResultSet rs) throws SQLException {
        ResultSetMetaData md = rs.getMetaData();
        int cols = md.getColumnCount();

        StringBuilder header = new StringBuilder();
        for (int c = 1; c <= cols; c++) {
            header.append(String.format("%-28s", md.getColumnLabel(c)));
        }
        System.out.println(header);
        System.out.println("-".repeat(header.length()));

        int rows = 0;
        while (rs.next()) {
            StringBuilder line = new StringBuilder();
```

```
            for (int c = 1; c <= cols; c++) {
                String value = rs.getString(c);
                line.append(String.format("%-28s", value == null ? "NULL" : value));
            }
            System.out.println(line);
            rows++;
        }
        if (rows == 0) {
            System.out.println("(no results)");
        }
    }
```

QueryMenu.java — `run` selects a query, `runQuery` binds the single parameter and executes it, `print` renders the `ResultSet` with aligned columns.

Error handling. All statements are prepared, so user input never reaches the SQL text. Database failures are caught and reported (`Database error: <message>`); multi-table writes roll back on failure (`Changes were rolled back.`); invalid menu input, non-numeric IDs, and malformed dates each print a specific message instead of terminating the program.

Appendix A — QueryMenu.java (full source)

```
package com.mycompany.foundit;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.SQLException;

/**
 * Runs the canned report queries.
 */
public final class QueryMenu {

    private static final String[] TITLES = {
        "Items in a category (WHERE)",
        "Items with location and reporter (JOIN)",
        "Items reported per reporter (JOIN + COUNT + GROUP BY)",
        "Found items never claimed (Subquery)",
        "Verified/unverified claims per validator (JOIN + SUM + GROUP BY)",
        "Items with more than one claim (GROUP BY + HAVING)",
        "Five most recently found items (ORDER BY)",
        "Average hours from found to claim, per category (AVG + GROUP BY)"
    };

    private static final String[] SQL = {
        "SELECT item_id, item_name, item_category, item_type FROM Item WHERE item_category = ?",

        "SELECT i.item_id, i.item_name, i.item_type, l.location_name, "
      + "CONCAT(u.first_name,' ',u.last_name) AS reporter "
      + "FROM Item i JOIN Location l ON l.location_id = i.location_id "
      + "JOIN User u ON u.user_id = i.reporter_id ORDER BY i.item_id",

        "SELECT u.user_id, CONCAT(u.first_name,' ',u.last_name) AS reporter, COUNT(i.item_id) AS
items_reported "
      + "FROM User u JOIN Item i ON i.reporter_id = u.user_id "
      + "GROUP BY u.user_id, u.first_name, u.last_name ORDER BY items_reported DESC",

        "SELECT i.item_id, i.item_name, f.found_at FROM Item i "
      + "JOIN FoundItem f ON f.item_id = i.item_id "
      + "WHERE i.item_id NOT IN (SELECT item_id FROM ClaimRequest)",

        "SELECT u.user_id, CONCAT(u.first_name,' ',u.last_name) AS validator, "
      + "SUM(c.request_type = 'VerifiedRequest') AS verified, "
      + "SUM(c.request_type = 'UnverifiedRequest') AS unverified "
      + "FROM ClaimRequest c JOIN User u ON u.user_id = c.validator_id "
      + "GROUP BY u.user_id, u.first_name, u.last_name",

        "SELECT i.item_id, i.item_name, COUNT(*) AS claim_count "
      + "FROM ClaimRequest c JOIN Item i ON i.item_id = c.item_id "
      + "GROUP BY i.item_id, i.item_name HAVING COUNT(*) > 1",

        "SELECT i.item_id, i.item_name, f.found_at FROM FoundItem f "
      + "JOIN Item i ON i.item_id = f.item_id ORDER BY f.found_at DESC LIMIT 5",

        "SELECT i.item_category, ROUND(AVG(TIMESTAMPDIFF(HOUR, f.found_at, c.request_date)),1) AS
avg_hours_to_claim "
      + "FROM ClaimRequest c JOIN FoundItem f ON f.item_id = c.item_id "
      + "JOIN Item i ON i.item_id = c.item_id GROUP BY i.item_category ORDER BY avg_hours_to_claim"
    };

    private final Connection conn;
    private final Console console;

    public QueryMenu(Connection conn, Console console) {
        this.conn = conn;
        this.console = console;
```

```
    }

    public void run() {
        while (true) {
            System.out.println("\n===== REPORTS / QUERIES =====");
            for (int i = 0; i < TITLES.length; i++) {
                System.out.printf("%d. %s%n", i + 1, TITLES[i]);
            }
            System.out.println("0. Back to main menu");

            int choice;
            try {
                choice = Integer.parseInt(console.readLine("Choose: "));
            } catch (NumberFormatException e) {
                System.out.println("Invalid input. Enter a number from 0 to " + TITLES.length + ".");
                continue;
            }

            if (choice == 0) {
                return;
            }
            if (choice < 1 || choice > TITLES.length) {
                System.out.println("Invalid choice. Enter a number from 0 to " + TITLES.length + ".");
                continue;
            }
            runQuery(choice - 1);
        }
    }

    private void runQuery(int idx) {
        try (PreparedStatement ps = conn.prepareStatement(SQL[idx])) {
            if (idx == 0) {
                String category = console.readLine("Category (e.g. Electronics, Clothing, Keys): ");
                if (category.isEmpty()) {
                    System.out.println("Category cannot be empty.");
                    return;
                }
                ps.setString(1, category);
            }

            try (ResultSet rs = ps.executeQuery()) {
                System.out.println("\n-- " + TITLES[idx] + " --");
                print(rs);
            }
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }

    private void print(ResultSet rs) throws SQLException {
        ResultSetMetaData md = rs.getMetaData();
        int cols = md.getColumnCount();

        StringBuilder header = new StringBuilder();
        for (int c = 1; c <= cols; c++) {
            header.append(String.format("%-28s", md.getColumnLabel(c)));
        }
        System.out.println(header);
        System.out.println("-".repeat(header.length()));

        int rows = 0;
        while (rs.next()) {
            StringBuilder line = new StringBuilder();
            for (int c = 1; c <= cols; c++) {
                String value = rs.getString(c);
                line.append(String.format("%-28s", value == null ? "NULL" : value));
            }
            System.out.println(line);
            rows++;
        }
        if (rows == 0) {
```

```
            System.out.println("(no results)");
        }
    }
}
```